

1. WHAT THIS SYSTEM IS

This repository ("EMA Leads Funnel") is the lead-capture funnel and admin CRM that feeds the separate main E-Migration Assist platform. The public brand shown to visitors is "E-Migration Assist"; the internal/operational identity is "EMA Leads Funnel". It is a sender: it captures leads, qualifies them, and (with consent) refers them onward. It is NOT the main EMA platform.

Major surfaces:

- Public assessment funnel (5-step assessment, optional document uploads, OTP verification, WhatsApp/email confirmation, public status lookup)
- Segment landing funnels (overstay, firm/business intake)
- Admin CRM back-office (leads, cases, kanban pipeline, dashboard, campaigns, templates, lifecycle rules, audit timeline)
- Referral tunnel (PII-safe, consent-gated hand-off to the main EMA platform)

2. TECHNOLOGY STACK

Frontend : React 18 + Vite SPA, Wouter routing, TanStack Query, shadcn/ui + Tailwind, TipTap rich-text composer
Backend : Express 5 on Node 24, TypeScript 5.9
Database : PostgreSQL with Drizzle ORM
Validation : Zod (server input/output), OpenAPI contract-first with Orval-generated React Query hooks
Background : pg-boss v12 job queue (campaign sends, scheduling) — requires a Reserved VM in production (autoscale breaks it)
Email : Resend (canonical sender noreply@emigration-assist.com, domain must be SPF/DKIM verified)
WhatsApp/SMS : Twilio (WhatsApp templates for OTP and confirmations)
File storage : Replit Object Storage (document uploads)
Monorepo : pnpm workspaces (artifacts/* apps, lib/* shared packages)
Testing : Playwright E2E suite (tests/e2e, scopes A-H) + docs pack

3. REPOSITORY LAYOUT

artifacts/emigration-assist/ React SPA (pages, components, theme)
artifacts/api-server/ Express API (routes/, lib/)
lib/db/src/schema/ Drizzle schema, one file per domain:
leads, leadCases, admin, imports,
campaigns, templates
lib/api-spec/openapi.yaml OpenAPI contract (Orval codegen source)
tests/ Playwright E2E suite + support helpers
docs/ Test plan, cases, data, defect log, results
ROADMAP.md Phase plan

4. DEVELOPMENT TOPOLOGY (REPLIT WORKSPACE)

Browser (preview pane)
|
v
Shared reverse proxy on localhost:80 (path-based routing)
|-- / -> Vite dev server (emigration-assist SPA)
|-- /api -> Express API server
|
v
PostgreSQL (DATABASE_URL) + pg-boss queue (same database)

In dev the SPA calls the API same-origin through the proxy.

5. PRODUCTION TOPOLOGY (LIVE, SPLIT DEPLOY)

Browser
|
v
www.emigration-assist.com/assessment/* (marketing site, Vercel,
| rewrite strips /assessment prefix separate project)
v

<assessment>.vercel.app/* (this repo's SPA build;
| fetch/XHR with credentials Vite base = /assessment/)
v
immigrationassist.replit.app/api/* (this repo's Express API +
pg-boss + PostgreSQL on a
Replit Reserved VM)

Cross-origin plumbing:

- API honours WEB_ORIGIN comma-separated CORS allow-list
- Admin session cookie flips to SameSite=None; Secure when CROSS_SITE_COOKIES=true; all admin fetches use credentials:"include"
- Fail-closed boot: CROSS_SITE_COOKIES=true without WEB_ORIGIN refuses to start
- Frontend resolves the API via VITE_API_URL (set on Vercel); unset in dev so same-origin BASE_URL is used
- Repo auto-pushes to GitHub on every checkpoint; Vercel auto-deploys on push

6. PUBLIC LEAD FLOW (END TO END)

1. Visitor lands on a funnel page (home, overstay, firm/business). Funnel context (route/theme) is stored on the lead as JSONB.
 2. 5-step assessment (dynamically 7/8 steps if documents are uploaded). Honeypot field ("website") silently rejects bots with a synthetic 201.
 3. Contact verification: OTP sent via WhatsApp first, falls back to email.
 4. Two-phase submission:
 - Lead row committed at the Terms step with finalize:false (nothing is sent yet)
 - POST /api/leads/:id/finalize triggers the confirmation email/WhatsApp, at most once (engagement row guard)
 5. Reference number is generated at insert but revealed only on the /thank-you page after finalize — never during the assessment.
 6. Visitor can later check progress via the unauthenticated /status page -> GET /api/public/status/:referenceNumber.
- Public APIs are rate-limited, PII-minimised, and validated server-side.

7. ADMIN CRM FLOW

Auth : email + password; opaque server-side session in an httpOnly cookie (7-day TTL); legacy x-admin-token fallback; superadmin user management with self-protection; forgot-password with single-use 1-hour hashed tokens.

Pipeline : 10-stage bidirectional lead status. One hard invariant: entering "converted" requires current status "ready_for_case" (enforced atomically; same PATCH creates the case). Case statuses are forward-only.

Lead ops : internal notes (append-only, on the audit trail), ownership (assigned_to), follow-ups with due dates, soft archive, atomic delete (blocked if a case exists).

Scoring : event-sourced (append-only lead_events + ~60s recompute worker); rubric points snapshotted per event.

Dashboard : segments, source attribution, saved views, kanban.

Audit : every privileged mutation writes an append-only lead_audit row (actor credential hashed; before/after hold codes and timestamps only, never raw PII/provider strings). Notes and portal/conversion verbs surface in a unified per-lead timeline.

8. CAMPAIGN / OUTREACH ENGINE

- Audience: zod-validated single-level AND/OR query compiler over a 12-field whitelist (32-rule cap), re-evaluated at fire time.
- Composer: TipTap rich text, DOMPurify-sanitised on write AND render.
- Sending: POST /send returns 202; pg-boss workers claim recipients atomically (2000 cap, single-winner finaliser); pause/resume and scheduled sends via a 30s scheduler tick.
- Merge tokens: first_name, full_name, reference, organization_name.
- Unsubscribe: HMAC-signed RFC-8058 one-click; unsubscribes table.
- Reusable comm_templates (channel locked after create; archived rows immutable).

9. CONVERSION, CASES, AND CLIENT PORTAL PREPARATION

- Lead -> case conversion is idempotent (ensureCaseForLead); the case

- snapshots the client-facing reference number.
- Workflow auto-attach assigns a workflow or flags review_required.
 - Client portal state machine on the case (portal_status):
 - not_prepared -> ready_to_activate (admin "Prepare Portal", gated on workflow assigned) -> activated (admin "Activate Portal", terminal).
- All transitions are single-transaction, SELECT ... FOR UPDATE, idempotent, audited exactly once. No client credentials or client-facing side effects exist yet — activation is a status flip for a future phase.

10. REFERRAL TUNNEL (SENDER SIDE ONLY)

- On explicit POPIA consent, a redacted referral is offered to a matched partner firm who accepts inside the separate main EMA platform.
- Byte-exact HMAC contract with two deliberate serializations (redirect token body vs key-sorted server-to-server body). Fail-closed: missing REFERRAL_TUNNEL_SECRET => 503.
- Structural PII invariant: the referrals table has NO name/email/phone columns; applicant PII travels only inside the signed push body.
- The "converted" callback is terminal and idempotent.

11. SECURITY POSTURE (KEY INVARIANTS)

- Server-side validation everywhere (Zod); document type allow-list is server-side only.
- Honeypot never rendered client-side; bots get a fake success.
- WhatsApp webhook fail-closed (503 without a signature secret).
- Audit rows never store raw credentials or provider error strings.
- Production boot refuses unsafe CORS/cookie combinations.
- Unsubscribe HMAC fails closed in production if no secret is set.
- Dev-only test bypasses (rate limit, OTP) refuse to activate when NODE_ENV=production.

12. TESTING AND QUALITY

- Playwright E2E suite in tests/e2e, scopes A-H: lead capture, qualification, outreach, conversion, pipeline visibility, negative paths, RBAC, auditability. Smoke subset via @smoke tag.
- No faked success: blocked cases are test.fixme with unlock conditions; two known product defects are tracked as expected failures (DEF-001 email format not validated; DEF-002 invalid WhatsApp silently dropped).
- Docs pack in /docs: test plan, test cases, test data, defect log, results template. Runbook in tests/README.md.

=====

END OF DOCUMENT

=====